Mudcard

- **I'm still a bit confused about the relationship between p_crit and the prediction outcome.**
  - a classification model returns predicted probabilities
  - those probabilities can be converted to a predicted class using p_crit
  - if you look at class 1 predicted probabilities in a binary classification problem, if the predicted probability is above p_cirt, you predict class 1; otherwise you predict class 0
- **Do we only use the P-R curve when the dataset is imbalanced?**
  - No
  - The way you should think about is that ROC should not be used on an imbalanced dataset.
  - P-R can be used on any datasest, balanced or imbalanced
- **The P-R curve doesn't start from 0 on the precision axis. Where does it start, and what determines that starting point? Is it the maximum/minimum of the curve? If so, isn't the rest of the plot range (e.g., precision below that point) basically useless?**
  - Go through Quiz 2 again
  - If p_crit = 0, TN = 0 and FN = 0 because all point are predicted to be class 1
  - In that case, TP and FP are some numbers.
  - TP is all the correctly predicted class 1 points, and FP is all the incorrectly predicted class 1 points, TP+FP = m where m is the total number of points
  - precision is TP/(TP+FP) or equivalently, TP/m, so precision in this case is the fraction points that are in class 1, the class balance of the dataset.
  - I wouldn't say that any part of a figure is useless :) it's just empty
- **Are there any cases where RMSE is preferred over MAE when choosing the evaluation metrics?**
  - Yes, think of how they measure the difference between the true and predicted values
  - With MAE, the difference is linear so small and large differences have the same weight
  - With RMSE, because we calclate the square of the differences, large errors contribute more to RMSE than small errors
  - So the RMSE is preferred if large errors in regression are especially costly
  - MAE is preferred if the cost of prediction error increases linearly with the error's magnitude

Overview of what we covered so far and what's coming

Complications in the ML pipeline:

- if your dataset is non-iid, address it in splitting
  - time series data: use VAR(p), and sklearn's TimeSeriesSplit
  - group structure: use the group based slitters
    - if there are categorical features in your dataset, it does not mean that there is group structure!
    - look for ID columns instead
- if a classification problem is imbalanced, address it in splitting
  - use stratified splitting methods
- if your dataset has missing values in categorical and ordinal features, address it in preprocessing
  - treat missing values as another category
  - do not impute, do not drop columns
- if dataset is imbalanced, use evaluation metrics that do not rely on the True Negative element of the confusion matrix
  - no accuracy, no ROC curve for example
  - use precision, recall, f score, p-r curve, etc instead
- if your dataset is imbalanced, be careful about oversampling and undersampling methods - Lecture 16
- missing values in continuous features: Lecture 18-19
- interpretability: Lecture 20-21

The supervised ML pipeline

0. Data collection/manipulation: you might have multiple data sources and/or you might have more data than you need

- you need to be able to read in datasets from various sources (like csv, excel, SQL, parquet, etc)
- you need to be able to filter the columns/rows you need for your ML model
- you need to be able to combine the datasets into one dataframe

1. Exploratory Data Analysis (EDA): you need to understand your data and verify that it doesn't contain errors

- do as much EDA as you can!

2. Split the data into different sets: most often the sets are train, validation, and test (or holdout)

- practitioners often make errors in this step!
- you can split the data randomly, based on groups, based on time, or any other non-standard way if necessary to answer your ML question

3. Preprocess the data: ML models only work if X and Y are numbers! Some ML models additionally require each feature to have 0 mean and 1 standard deviation (standardized features)

- often the original features you get contain strings (for example a gender feature would contain 'male', 'female', 'non-binary', 'unknown') which needs to be transformed into numbers
- often the features are not standardized (e.g., age is between 0 and 100) but it needs to be standardized

4. Choose an evaluation metric: depends on the priorities of the stakeholders

- often requires quite a bit of thinking and ethical considerations

5. Choose one or more ML techniques: it is highly recommended that you try multiple models

- start with simple models like linear or logistic regression
- try also more complex models like nearest neighbors, support vector machines, random forest, etc.

6. Tune the hyperparameters of your ML models (aka cross-validation or hyperparameter tuning)

- ML techniques have hyperparameters that you need to optimize to achieve best performance
- for each ML model, decide which parameters to tune and what values to try
- loop through each parameter combination
  - train one model for each parameter combination
  - evaluate how well the model performs on the validation set
- take the parameter combo that gives the best validation score
- evaluate that model on the test set to report how well the model is expected to perform on previously unseen data

7. Interpret your model: black boxes are often not useful

- check if your model uses features that make sense (excellent tool for debugging)
- often model predictions are not enough, you need to be able to explain how the model arrived to a particular prediction (e.g., in health care)

Non-linear ML algorithms

By the end of this lecture, you will be able to

- Summarize the no free lunch theorem
- Describe how decision trees work
- Describe how a random forest works

Non-linear ML algorithms

By the end of this lecture, you will be able to

- **Summarize the no free lunch theorem**
- Describe how decision trees work
- Describe how a random forest works

Which ML algorithm to try on your dataset?

- **No supervised ML algorithm is universally better than all others** - no free lunch theorem
- **For every learning algorithm, there is a dataset/task on which it fails. Even though another algorithm can successfully learn on it.**
- If we had a universally good algorithm, we all just would use that one algorithm.
- you need to try as many as you can to find the one that performs best
- you will see during the final project presentations that different models perform best for different datasets

Quiz 1

Non-linear ML algorithms

By the end of this lecture, you will be able to

- Summarize the no free lunch theorem
- **Describe how decision trees work**
- Describe how a random forest works

Decision trees

- no free lunch theorem in action!

- deep learning tools are all the rage nowadays but tree-based methods outperform deep learning tools on tabular data
- see NeurIPS paper here
- we discuss decision trees and random forest today, gradient boosting and XGBoost on Monday

- Decision tree: the data is split according to certain features
- Here is an example tree fitted to data:



- Trees have nodes and leaves.
  - Nodes are where the split happens
  - Leaves are where we predict the target variable
- The depth of the decision tree is the number of nodes along the longest path from the root node to a leaf.
  - The tree above has a depth of 2.
- Notice that each split is an if-else statement (binary split), and only one feature is split on each node
  - splits with three or more branches or complex conditions (i.e., split over multiple features in one node) are not done.
- The critical values and features in the nodes are determined automatically by minimizing a cost function.

Non-linear ML algorithms

By the end of this lecture, you will be able to

- Summarize the no free lunch theorem
- Describe how decision trees work
- **Describe how a random forest works**

Random forest

- Random forest: ensemble of independent random decision trees
- Each tree sees a random subset of the training data, that's why the forest is random.
  - each tree is trained on a random subset of features or a random subset of points, or both
- Majority voting among the trees determines the final predicted class
- the fraction of votes is used to calculate predicted probabilities



Quiz 2

- **Use the dataset below and create a decision tree with max_depth = 2 to predict the target variable!**
- **What is your tree's prediction for each person?**
- Use the features that are assigned to your team!
  - You do not have to use all of the assigned features.
  - You cannot use any feature that was not assigned to your team.
- Remember, you might not be able to find a tree that predicts all points perfectly.
- It just needs to get as many points as possible right.
- Each split in the tree is based on one feature and one critical value.
- Do not use and/or statements in the condition.

In [15]:
```
import numpy as np
# which features should your team use?
ftrs_to_use = np.random.randint(1,7,size=3)
print('use columns', np.unique(ftrs_to_use), 'to create your decision tree.'
```

```
use columns [3 4 5] to create your decision tree.
```

Person	Age	Gender (M=0, F=1)	Is student?	Is parent?	Uses computer for work?	Hours/day on computer	Likes computer games?
Person 0	62	0	0	1	1	1.0	**1**
Person 1	36	1	0	1	1	2.5	**1**
Person 2	33	1	0	1	1	2.5	**0**
Person 3	12	0	1	0	0	5.0	**0**
Person 4	85	1	0	1	0	1.0	**0**
Person 5	23	0	1	0	0	5.0	**1**
Person 6	18	0	1	0	0	5.0	**1**
Person 7	5	1	1	0	0	0.0	**0**
Person 8	10	1	1	0	0	0.0	**1**
Person 9	48	0	0	1	1	2.5	**0**

The predictions of the trees

Correct label	Tree 1	Tree 2	Tree 3	Tree 4	Tree 5	Random Forest
1						
1						
0						
0						
0						
1						
1						
0						
1						
0						
Accuracy						

Mud card

In [ ]: